Bahasa Pemrograman Tingkat Rendah (MWCA010)

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

Operasi Logika, Pergeseran, dan Rotasi

Pertemuan 10

Dosen Pengampu

Cecep Suwanda, S.Si., M.Kom.

Semester Ganjil

Capaian dan Tujuan Pembelajaran

Sub-CPMK 3.2

Menerapkan operasi logika *bitwise* serta operasi pergeseran dan rotasi (`AND`, `OR`, `XOR`, `NOT`, `TEST`, `SHL`, `SHR`, `SAL`, `SAR`, `ROL`, `ROR`) untuk manipulasi bit.

CPMK yang didukung: CPMK03 ▪ **CPL yang didukung: CPL03**

Setelah pertemuan ini, mahasiswa mampu:

1. menjelaskan tabel kebenaran `AND`, `OR`, `XOR`, dan `NOT`;
2. menjelaskan pengaruh operasi logika terhadap `CF`, `OF`, `ZF`, `SF`, dan `PF`;
3. membedakan `TEST` dan `CMP` serta memilih yang tepat;
4. menerapkan penyalakan, pemadaman, pembalikan, dan pengujian bit ke-n pada sebuah byte;
5. membedakan pergeseran logis, pergeseran aritmetika, dan rotasi beserta peran `CF`;
6. menggantikan perkalian atau pembagian pangkat dua dengan pergeseran;
7. menyusun program manipulasi bit: paritas, *packing*/*unpacking* BCD, dan *XOR cipher*.

Peta Materi

1. **Operasi Logika pada Level Bit**
   tabel kebenaran; pengaruh terhadap *flag*; *masking* dan manipulasi bit ke-*n*; `TEST` vs `CMP`
2. **Pergeseran dan Rotasi**
   `SHL`/`SHR` dan `SAL`/`SAR`; `ROL`, `ROR`, `RCL`, `RCR`; perkalian dan pembagian cepat
3. **Studi Kasus Manipulasi Bit**
   pemeriksaan paritas; *packing* dan *unpacking* BCD; *XOR cipher* sederhana

Praktik pada pertemuan ini

Menelusuri `paritas.asm` dengan *Single Step*, mencatat `AL`, `BL`, dan `CF`; dilanjutkan manipulasi bit ke-*n* serta *packing* BCD. Ditutup dengan **Kuis 3**.

BAGIAN

Operasi Logika pada Level Bit

Operasi Logika pada Level Bit

Berbeda dari `ADD` dan `SUB` yang menjalarkan *carry* antarbit, operasi logika memperlakukan setiap posisi bit sebagai satuan mandiri yang tidak memengaruhi tetangganya.

Bit A	**Bit** B	`AND`	`OR`	`XOR`
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Hasilnya hanya bergantung pada pasangan bit yang bersesuaian; keempat instruksi bekerja pada operand 8 bit maupun 16 bit.

Sumber: Bab XI, Tabel Kebenaran dan Empat Instruksi Dasar

Sifat Empat Instruksi Logika

- `AND`: 1 hanya bila kedua bit 1 — mempertahankan bit tertentu, memadamkan sisanya.
- `OR`: 1 bila salah satu bit 1 — menyalakan bit tanpa mengubah bit lain.
- `XOR`: 1 bila kedua bit berbeda — membalik bit.
- `NOT`: membalik seluruh bit dan *tidak* menyentuh *flag*.

```
MOV AL, 10100101b   ; AL = A5h
AND AL, 00001111b   ; AL = 05h
MOV BL, 11000011b   ; BL = C3h
OR  AL, BL          ; AL = C7h
MOV CL, 10100101b   ; CL = A5h
XOR CL, 11111111b   ; CL = 5Ah
NOT CL              ; CL = A5h
```

Sumber: Bab XI, Tabel Kebenaran dan Empat Instruksi Dasar

Pengaruh Operasi Logika terhadap *Flag*

Flag	Nilai setelah operasi logika	Alasan
`CF`	Selalu 0	Tidak ada *carry* yang keluar dari bit paling kiri
`OF`	Selalu 0	Operasi *bitwise* tidak menghasilkan luapan bertanda
`ZF`	1 bila seluruh bit hasil bernilai 0	Memeriksa apakah suatu bidang bit kosong
`SF`	Menyalin bit paling kiri hasil	Hasil dianggap negatif bila ditafsirkan bertanda
`PF`	1 bila cacah bit 1 pada byte rendah genap	Dipakai pada pemeriksaan paritas
`AF`	Tidak terdefinisi	Jangan diandalkan setelah operasi logika

`NOT` satu-satunya pengecualian: tidak menyentuh *flag* sama sekali, sehingga `ZF`, `SF`, dan `PF` masih bernilai seperti sebelumnya.

Sumber: Bab XI, Pengaruh Operasi Logika terhadap Flag

Aturan Penulisan Operand

Aturan legalitas operand sama dengan `MOV` dan `ADD`: operand tujuan boleh register atau memori, operand sumber boleh register, memori, atau nilai segera.

```
MOV AL, nilai           ; DB + AL -> ukuran sudah sama
MOV AX, WORD PTR total  ; DW + AX -> WORD PTR menyesuaikan ukuran
AND AL, BYTE PTR nilai  ; AL (8 bit) dengan operand memori
ADD BL, CL              ; register dengan register
```

Dua larangan yang tetap berlaku

Memori ke memori tidak diizinkan sehingga perlu register perantara, dan kedua operand wajib berukuran sama.

Sumber: Bab XI, Aturan Penulisan Operand

Manipulasi Bit ke-n dengan *Mask*

Misalkan AL berisi A5h = 1010 0101b dan bit yang disasar bit ke-3 (08h). *Mask* adalah 2^n: satu bit menyala pada posisi yang dituju.

Kehendak	**Mask**	**Instruksi**	**Hasil pada A5h, bit 3**
Menyalakan bit	mask	OR AL, mask	1010 1101b = ADh
Memadamkan bit	NOT mask	AND AL, mask	1010 0101b = A5h
Membalik bit	mask	XOR AL, mask	1010 1101b = ADh
Menguji bit	mask	TEST AL, mask	ZF = 0 berarti bit 3 menyala

```
OR   AL, 08h        ; nyalakan bit 3      (A5h -> ADh)
AND  AL, 0F7h       ; padamkan bit 3      (F7h = NOT 08h)
XOR  AL, 08h        ; balikkan bit 3
TEST AL, 08h        ; uji bit 3 tanpa mengubah AL
```

Sumber: Bab XI, Penerapan Praktis: Manipulasi Bit Tertentu

Memadamkan Bit: Kesalahan yang Sering Terjadi

Nilai mask sering ditulis terbalik

Untuk memadamkan bit ke-3 yang benar adalah `AND AL, 0F7h` — bit ke-3 pada mask bernilai 0. Penulisan `AND AL, 08h` justru mempertahankan bit itu dan menghapus seluruh bit lain.

Mask penghapus dihitung saat berjalan

Mask disimpan ke register, dibalik dengan `NOT`, lalu dipakai pada `AND`. Karena `NOT` tidak menyentuh *flag*, keputusan lompatan sebelumnya tetap sah.

Kesalahan kedua: menganggap `AND` menyetel `CF`. Operasi logika selalu mengosongkan `CF` dan `OF`.

Sumber: Bab XI, Penerapan Praktis: Manipulasi Bit Tertentu

TEST dan CMP: Dua Instruksi Penguji

Keduanya melakukan operasi tanpa menyimpan hasilnya; yang tertinggal hanya jejak pada *flag*.

Aspek	TEST	CMP
Operasi internal	AND tanpa menyimpan hasil	SUB tanpa menyimpan hasil
CF dan OF	Selalu 0	Sesuai hasil pengurangan
Pemakaian utama	Memeriksa keadaan bit tertentu	Membandingkan besar dua bilangan secara utuh
Lompatan lazim	JZ/JNZ, JP/JNP	JE/JNE, JA/JB, JG/JL
Contoh	TEST AL, 01h lalu JNZ	CMP AL, BL lalu JA

Kebiasaan khusus: TEST AL, AL tidak mengubah isi AL, tetapi memperbarui ZF, SF, dan PF; pola ini menggantikan CMP AL, 0 dengan kode dua byte lebih pendek.

Sumber: Bab XI, TEST dan CMP: Dua Instruksi Penguji

Rangkuman Bagian 1

- Operasi logika memandang setiap bit sebagai satuan mandiri; aturannya mengikuti gerbang logika dasar dan tidak menjalarkan *carry*.
- `AND` untuk *masking*, `OR` untuk menyalakan, `XOR` untuk membalik; `NOT` membalik seluruh bit dan tidak menyentuh *flag*.
- `CF` dan `OF` selalu 0 setelah `AND`, `OR`, `XOR`, dan `TEST`, sedangkan `ZF`, `SF`, dan `PF` mengikuti hasil — sumber kesalahan bila `CF` dipercaya.
- Bit ke-n disentuh dengan mask 2^n: nyalakan dengan `OR`, padamkan dengan `AND` memakai komplemen mask, balikkan dengan `XOR`, uji dengan `TEST`.
- `TEST` = `AND` tanpa menyimpan hasil (memeriksa bit); `CMP` = `SUB` tanpa menyimpan hasil (membandingkan nilai utuh).

BAGIAN

Pergeseran dan Rotasi

Pergeseran Logis: SHL dan SHR

Operasi logika bekerja pada posisi bit yang tetap; pergeseran justru memindahkan isi bit ke posisi lain. Bit yang terdesak keluar ditampung pada CF.

```
SHL AL, 1           ; geser AL satu bit ke kiri, bit kanan diisi 0
SHR AX, 1           ; geser AX satu bit ke kanan, bit kiri diisi 0
MOV CL, 4           ; jumlah pergeseran disimpan pada CL
SHL BX, CL          ; geser BX empat bit ke kiri
```

Jumlah pergeseran

Ditulis sebagai angka 1 atau disimpan pada CL. Pada 8086 nilai CL dipakai apa adanya (sampai 255), sedangkan pada 80286 ke atas hanya lima bit terendahnya (batas 31).

Sumber: Bab XI, Pergeseran Logis: SHL dan SHR

Pergeseran Aritmetika: SAL dan SAR

- SAL (*Shift Arithmetic Left*) secara operasi *identik* dengan SHL dan menghasilkan kode mesin yang sama; namanya hanya menegaskan nilai dianggap bertanda.
- SAR berbeda secara hakiki: bit yang masuk dari sisi kiri bukan nol, melainkan *salinan bit tanda*, sehingga tanda bilangan terpelihara.

Mengapa SAR diperlukan

Pada 1111 1111b (-1 dalam komplemen dua), SHR satu bit memberi 0111 1111b ($+127$) karena bit tanda terisi nol. Bagi bilangan bertanda negatif, hanya SAR yang benar.

Sumber: Bab XI, Pergeseran Aritmetika: SAL dan SAR

Perbedaan Pergeseran pada Byte 96h



SHR memberi 0100 1011b (+75) karena bit tanda dipaksa nol, sedangkan SAR memberi 1100 1011b (−53) karena bit tanda disalin ulang.

Sumber: Bab XI, Pergeseran Aritmetika: SAL dan SAR

Ringkasan Kedelapan Instruksi Geser dan Rotasi

Instruksi	Arah	Bit yang masuk	Flag terpengaruh
SHL	kiri	0 di bit paling kanan	CF menerima bit paling kiri; SF, ZF, PF ikut hasil
SAL	kiri	0 di bit paling kanan	sama dengan SHL (kode mesin identik)
SHR	kanan	0 di bit paling kiri	CF menerima bit paling kanan; SF, ZF, PF ikut hasil
SAR	kanan	salinan bit tanda	CF menerima bit paling kanan; SF tetap sesuai tanda
ROL	kiri	bit keluar dari kiri	CF menerima bit berputar; OF benar bila jumlah 1
ROR	kanan	bit keluar dari kanan	CF menerima bit berputar; OF benar bila jumlah 1
RCL	kiri	CF lama	CF menerima bit paling kiri, lalu ikut berputar
RCR	kanan	CF lama	CF menerima bit paling kanan, lalu ikut berputar

Pada 8086 OF hanya terdefinisi benar bila jumlah pergeseran satu; CF selalu berisi bit terakhir yang keluar, berapa pun jumlah pergeserannya.

Sumber: Bab XI, Ringkasan Kedelapan Instruksi Geser dan Rotasi

Rotasi: ROL dan ROR

Rotasi memindahkan bit secara melingkar: bit yang terdesak keluar dari satu sisi masuk kembali ke sisi lain, sehingga tidak ada informasi yang hilang. Bit yang berputar selalu melewati CF.

```
MOV AL, 10000001b    ; AL = 81h
ROL AL, 1            ; AL = 00000011b, CF = 1 (bit 7 pindah ke bawah)

MOV AL, 10000001b    ; AL = 81h
ROR AL, 1            ; AL = 11000000b, CF = 1 (bit 0 pindah ke atas)
```

CF merefleksikan bit terakhir yang melewatinya, bukan bit yang hilang seperti pada pergeseran.

Sumber: Bab XI, Rotasi: ROL, ROR, RCL, dan RCR

RCL, RCR, dan Pergeseran 32 Bit

Pada RCL, CF menjadi bit tambahan di sebelah kiri operand: bit paling kiri masuk ke CF, sedangkan nilai CF yang lama masuk ke bit paling kanan. RCR berlaku kebalikannya.

```
CLC                 ; CF = 0
MOV AL, 10000001b   ; AL = 81h
RCL AL, 1           ; AL = 00000010b, CF = 1 (CF lama masuk ke bit 0)

; menggeser bilangan 32 bit (DX:AX) satu bit ke kanan
SAR DX, 1           ; word tinggi digeser, bit tanda disalin
RCR AX, 1           ; bit yang keluar dari DX masuk ke AX
```

Pasangan SAR dan RCR memindahkan bit dari word tinggi ke word rendah sehingga bilangan 32 bit dapat digeser sebagai satu kesatuan.

Sumber: Bab XI, Rotasi: ROL, ROR, RCL, dan RCR

Perkalian dan Pembagian Cepat dengan Pergeseran

Pergeseran satu bit setara dengan mengalikan atau membagi dengan dua, sehingga pangkat dua tidak perlu memakai `MUL` atau `DIV`.

```
MOV AX, 5
MOV CL, 4           ; 16 = 2 pangkat 4
SHL AX, CL          ; AX = 80  (5 x 16)

MOV AX, 5
MOV BX, AX          ; BX = 5, salinan untuk 5 x 8
SHL AX, 1           ; AX = 10  (5 x 2)
MOV CL, 3
SHL BX, CL          ; BX = 40  (5 x 8)
ADD AX, BX          ; AX = 50  (10 + 40)
```

Bentuk terakhir memakai $5 \times 10 = (5 \times 2) + (5 \times 8)$; `SHL` dan `ADD` hanya perlu beberapa siklus, sedangkan `MUL` pada 8086 lebih dari seratus siklus.

Sumber: Bab XI, Perkalian dan Pembagian Cepat dengan Pergeseran

Peringatan: Membagi Bilangan Bertanda

Jangan memakai SHR untuk bilangan bertanda

`SHR AX, 1` mengosongkan bit tanda, sehingga -1 (`FFFFh`) menjadi `7FFFh` ($+32767$). Gunakan `SAR`, bukan `SHR`.

Namun SAR tidak membulatkan ke arah nol

`SAR AX, 1` pada -1 menghasilkan -1, bukan 0. Instruksi `IDIV` tetap diperlukan bila hasil pembagian harus dibulatkan ke arah nol.

Pergeseran kanan logis hanya sah untuk pembagian bilangan tak bertanda.

Sumber: Bab XI, Perkalian dan Pembagian Cepat dengan Pergeseran

Rangkuman Bagian 2

- SHL dan SAL identik (kode mesin sama) dan menyisipkan 0 dari kanan; SHR menyisipkan 0 dari kiri; SAR menyisipkan salinan bit tanda.
- Jumlah pergeseran ditulis 1 atau disimpan pada CL. CF selalu berisi bit terakhir yang keluar; OF hanya terdefinisi benar untuk jumlah satu.
- ROL/ROR memutar bit secara melingkar tanpa kehilangan informasi, sedangkan RCL/RCR menyertakan CF di ujung lingkaran putar.
- Pasangan SAR dan RCR menggeser bilangan 32 bit (DX:AX) sebagai satu kesatuan.
- Geser satu bit setara kali atau bagi dua: SHL bersama ADD menggantikan MUL yang boros siklus, tetapi pembagian bertanda hanya sah dengan SAR.

BAGIAN

Studi Kasus Manipulasi Bit

Memeriksa Paritas Sebuah Byte

Paritas menyatakan apakah cacah bit 1 sebuah byte genap atau ganjil; pemeriksaan ini mendeteksi satu bit yang berubah saat transmisi dan dilayani `PF`.

Cara	Instruksi kunci	Catatan
Flag `PF`	`TEST AL, AL` lalu `JP`	Paling singkat; hanya untuk byte rendah hasil operasi
Pelipatan `XOR`	`XOR AL, AH` tiga kali	Tidak bergantung `PF`; berlaku untuk *word*/*dword*
Pergeseran `ADC`	`SHR AL, 1` lalu `ADC BL, 0`	Paling gamblang; iterasi sebanyak cacah bit operand

Byte uji `1011 0110b` memiliki lima bit 1 — cacah ganjil, sehingga program mencetak `GANJIL`.

Sumber: Bab XI, Memeriksa Paritas Sebuah Byte

Paritas Tanpa PF: Pelipatan XOR

Byte dilipat menjadi setengah lebarnya dengan XOR, lalu diulang sampai tersisa satu bit.

```
MOV AL, nilai       ; byte yang diperiksa
MOV AH, AL
SHR AH, 4
XOR AL, AH          ; lipat nibble tinggi ke nibble rendah
MOV AH, AL
SHR AH, 2
XOR AL, AH
MOV AH, AL
SHR AH, 1
XOR AL, AH
AND AL, 1           ; 1 = cacah bit 1 ganjil, 0 = genap
```

Pelipatan berturut-turut 4, 2, lalu 1 bit memindahkan seluruh bit ke posisi terendah, sehingga AND AL, 1 menyisakan paritasnya.

Sumber: Bab XI, Memeriksa Paritas Sebuah Byte

Packing dan *Unpacking* Dua Digit BCD

BCD menyimpan bilangan desimal empat bit per digit: satu byte menampung puluhan pada nibble tinggi dan satuan pada nibble rendah.

Operasi	Instruksi	Hasil
Packing	`MOV AL, puluhan` lalu `SHL AL, CL` (CL = 4) lalu `OR AL, satuan`	`0111 0101b` = `75h`
Ambil satuan	`AND AL, 0Fh`	`0000 0101b` = `05h`
Ambil puluhan	`SHR AL, CL` dengan CL = 4	`0000 0111b` = `07h`

```
MOV AL, 7           ; digit puluhan
MOV CL, 4
SHL AL, CL          ; AL = 70h (0111 0000b)
OR  AL, 5           ; AL = 75h -> 75 dalam BCD
MOV AH, AL
AND AH, 0Fh         ; AH = 05h, digit satuan
SHR AL, CL          ; AL = 07h, digit puluhan
```

Sumber: Bab XI, Packing dan Unpacking Dua Digit BCD

XOR Cipher Sederhana

Bila A di-XOR dengan kunci K, hasilnya $B = A \oplus K$. Nilai asli dipulihkan dengan meng-XOR-kan B sekali lagi dengan kunci yang sama: $B \oplus K = (A \oplus K) \oplus K = A$.

```
LEA SI, asli          ; SI <- penunjuk sumber
LEA DI, buffer        ; DI <- penunjuk tujuan
ENKRIPSI:
MOV AL, [SI]          ; ambil satu byte
CMP AL, '$'           ; penanda akhir string?
JE  TAMPIL_ENK
XOR AL, kunci         ; XOR dengan kunci 20h
MOV [DI], AL          ; simpan hasil
INC SI
INC DI
JMP ENKRIPSI
```

Keluaran

```
Teks asli   : BELAJARASM
Terenkripsi : belajarasm
Terdekripsi : BELAJARASM
```

Dekripsi memakai perulangan yang sama, tetapi menulis kembali di tempat yang sama. Kunci `20h` dipilih agar hasil enkripsi tetap terbaca.

Sumber: Bab XI, XOR Cipher Sederhana

Rangkuman Bagian 3

- Paritas byte ditentukan oleh cacah bit 1: `TEST AL, AL` lalu `JP`, pelipatan `XOR` tiga kali, atau `SHR` delapan kali dengan `ADC BL, 0`.
- Pelipatan `XOR` tidak bergantung pada `PF` dan mudah diperluas ke word maupun *dword*.
- *Packing* BCD menyusun dua digit dengan `SHL` lalu `OR`; *unpacking* memisahkannya dengan `AND 0Fh` dan `SHR` empat bit.
- *XOR cipher* dapat dibalik karena $A \oplus K \oplus K = A$, sehingga kunci yang sama dipakai untuk enkripsi dan dekripsi.
- Namun cara itu *bukan* pengamanan sungguh-sungguh: bila sebagian teks dapat ditebak, kuncinya terhitung dengan satu operasi `XOR`.

Contoh 1: Memeriksa Paritas Byte

```
MOV AL, nilai        ; byte yang diperiksa
MOV BL, 0            ; penghitung cacah bit 1
MOV CX, 8            ; delapan bit diperiksa
ULANG:
SHR AL, 1            ; bit terkanan pindah ke CF
ADC BL, 0            ; BL = BL + CF
LOOP ULANG           ; ulangi sampai CX habis
TEST BL, 1           ; uji bit terendah cacah bit 1
JNZ CETAK_GANJIL     ; ZF = 0 -> cacah ganjil
MOV AH, 09h          ; cacah genap: cetak GENAP
LEA DX, genap
INT 21h
JMP SELESAI
CETAK_GANJIL:        ; cacah ganjil: cetak GANJIL
```

Keluaran

```
Byte 10110110b berparitas: GANJIL
```

`ADC BL, 0` menjumlahkan penghitung dengan `CF` — di sini *carry* dipakai sebagai sumber bit, bukan sisa penjumlahan.

Byte `1011 0111b` akan mencetak `GENAP`.

`code/asm/paritas.asm`

Contoh 2: Enkripsi dan Dekripsi *XOR Cipher*

```
LEA SI, asli          ; SI <- penunjuk teks asli
LEA DI, buffer        ; DI <- penunjuk penampung
ENKRIPSI:
MOV AL, [SI]          ; ambil satu byte teks asli
CMP AL, '$'           ; sudah sampai penanda akhir?
JE  TAMPIL_ENK        ; ya -> hentikan perulangan
XOR AL, kunci         ; XOR dengan kunci 20h
MOV [DI], AL          ; simpan byte terenkripsi
INC SI                ; maju ke byte berikutnya
INC DI
JMP ENKRIPSI
; DEKRIPSI memakai perulangan sama, XOR ulang di
     tempat yang sama
```

Keluaran

```
Teks asli   : BELAJARASM
Terenkripsi : belajarasm
Terdekripsi : BELAJARASM
```

Perulangan berhenti pada penanda '$', sehingga panjang string tidak perlu dihitung.

`code/asm/xor-cipher.asm`

Contoh 3: Memanipulasi Bidang Bit Status

Satu byte menampung beberapa keadaan sekaligus: bit 0 lampu menyala, bit 3 pintu terbuka, bit 5 alarm aktif.

```
.DATA
    status  DB 00000000b        ; semula seluruh bit padam
.CODE
    OR  status, 00100001b       ; nyalakan lampu (bit 0) + alarm (bit 5)
    AND status, 11011111b       ; padamkan alarm (bit 5)
    XOR status, 00001000b       ; balikkan pintu (bit 3) -> terbuka
    XOR status, 00001000b       ; balikkan lagi -> tertutup
    TEST status, 00001000b      ; uji bit 3, AL tidak berubah
    JZ  PINTU_TERTUTUP          ; ZF = 1 -> pintu tertutup
```

Jauh lebih hemat daripada satu variabel per keadaan: satu byte menampung delapan keadaan dan pemeriksaannya cukup satu `TEST` diikuti lompatan bersyarat.

Sumber: Bab XI, Contoh 3: Memanipulasi Bidang Bit Status

Praktikum: Manipulasi Bit

Judul praktikum

Manipulasi bit — pemeriksaan paritas dan *packing* BCD ▪ **Sub-CPMK:** 3.2 ▪ **Waktu:** 100 menit ▪ **Bentuk:** individu

Alat dan bahan

- Komputer bersistem operasi Windows
- Emu8086 atau GUI Turbo Assembler + TASM dan TLINK
- Berkas `paritas.asm` dan `xor-cipher.asm`
- Lembar pengamatan paritas

Langkah kerja

1. Telusuri `paritas.asm` dengan `Single Step`; catat `AL`, `BL`, `CF`.
2. Uji byte `1011 0111b` dan `0101 0101b`.
3. Nyalakan, padamkan, balikkan, lalu uji bit ke-3 `A5h`.
4. Susun BCD 75, pisahkan kembali, ulangi untuk 93.
5. Bandingkan `SHR` dan `SAR` pada `FF96h`; hitung 7×12 tanpa `MUL`.

Dikumpulkan: tabel pengamatan paritas, program manipulasi bit ke-n, program *packing* BCD beserta tangkapan layar, tabel `SHR` vs `SAR`, dan lembar **Kuis 3**.

Latihan

1. Jelaskan dengan kalimat Anda sendiri perbedaan cara kerja `AND`, `OR`, dan `XOR` pada level bit. Sertakan tabel kebenaran singkat untuk masing-masing instruksi.

2. Mengapa `CF` dan `OF` selalu bernilai 0 setelah `AND`, `OR`, `XOR`, atau `TEST`? Kaitkan dengan sifat operasi *bitwise* yang tidak menjalarkan *carry* antarbit.

3. Sebutkan tiga perbedaan `TEST` dan `CMP`, lalu tentukan yang lebih tepat untuk memeriksa apakah bit ke-4 `AL` menyala, dan untuk memeriksa apakah `AX` lebih besar daripada `BX`.

4. Diketahui `AL` = `A5h`. Tentukan nilai akhir `AL` dan nilai `ZF` untuk potongan program berikut: `AND AL, 00001111b`; `OR AL, 00110000b`; `XOR AL, 00000011b`; `TEST AL, AL`.

5. Diketahui `AL` = `1001 0110b` (`96h`) yang ditafsirkan bertanda. Tentukan hasil dan nilai `CF` untuk `SHL AL, 1`, `SHR AL, 1`, dan `SAR AL, 1`; jelaskan mengapa hanya `SAR` yang memberi hasil pembagian bertanda yang benar.

Rangkuman

- `AND`, `OR`, `XOR`, dan `NOT` bekerja pada setiap posisi bit secara mandiri tanpa menjalarkan *carry*: `AND` untuk *masking*, `OR` untuk menyalakan, `XOR` untuk membalik, dan `NOT` membalik seluruh bit tanpa menyentuh *flag*.
- Setelah `AND`, `OR`, `XOR`, dan `TEST`, `CF` dan `OF` selalu 0, sedangkan `ZF`, `SF`, dan `PF` mengikuti hasil.
- Manipulasi bit ke-n memakai mask 2^n: `OR` untuk menyalakan, `AND` dengan komplemen mask untuk memadamkan, `XOR` untuk membalik, `TEST` untuk menguji.
- `TEST` = `AND` tanpa menyimpan hasil (memeriksa bit); `CMP` = `SUB` tanpa menyimpan hasil (membandingkan nilai utuh).
- `SAL` identik dengan `SHL`, sedangkan `SAR` menyisipkan salinan bit tanda; `ROL`/`ROR` memutar melingkar dan `RCL`/`RCR` menyertakan `CF`. Bit yang keluar selalu ditampung `CF`.
- Tiga studi kasus: pemeriksaan paritas (`PF`, pelipatan `XOR`, atau `SHR` + `ADC`), *packing* BCD, dan *XOR cipher* yang dapat dibalik karena $A \oplus K \oplus K = A$.

Cek Pemahaman

1. Nilai `CF` dan `OF` setelah instruksi `XOR AL, BL` dijalankan adalah ...
   - 1.1 keduanya 0
   - 1.2 keduanya 1
   - 1.3 `CF` = 1 dan `OF` = 0
   - 1.4 bergantung pada nilai `AL` dan `BL`

2. Instruksi untuk memadamkan bit ke-3 `AL` tanpa mengubah bit lain adalah ...
   - 2.1 `OR AL, 08h`
   - 2.2 `AND AL, 08h`
   - 2.3 `AND AL, 0F7h`
   - 2.4 `XOR AL, 08h`

3. Diketahui `AL` = `1111 1111b` yang ditafsirkan sebagai -1. Instruksi `SHR AL, 1` menghasilkan nilai ...
   - 3.1 -1 karena bit tanda dipertahankan
   - 3.2 $+127$ karena bit tanda diisi nol
   - 3.3 0 karena seluruh bit bergeser keluar
   - 3.4 -64 karena hasilnya dibulatkan ke bawah

Penutup

Sebelum pertemuan berikutnya

- Selesaikan praktikum: tabel pengamatan paritas, program manipulasi bit ke-n, program *packing* BCD, dan tabel perbandingan `SHR` dengan `SAR`.
- Latih kembali penelusuran bit secara manual pada byte `96h` dan `A5h` sebelum Kuis 3.
- Baca **Bab XII** — Struktur Kontrol dan Percabangan.

Bacaan lanjutan: Bab XI bagian Ringkasan Kedelapan Instruksi Geser dan Rotasi serta Studi Kasus Manipulasi Bit; daftar *encoding* dan waktu siklus Intel 8086 (perbandingan `SHL` dan `MUL`); pembahasan *flag* secara mendalam pada Bab V.

Terima kasih

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil